

STAGE M1 MIAGE — 2025 / 2026

Plasti Matic

Plateforme interne (tableau de bord) & boutique en ligne
pour EURL Plasti Matic

2

APPLICATIONS

1

API REST

25+

ENDPOINTS

36

TESTS
AUTOMATISÉS

EURL Plasti Matic — Bordj Bou Arreridj, Algérie

Fabricant de vêtements de travail, équipements de sécurité et chaussures industrielles

Document de préparation à la soutenance

Sommaire

1. Contexte et objectifs	Entreprise, cahier des charges
2. Architecture technique	Stack, monorepo, modèle de données
3. Le tableau de bord (dashboard)	Catalogue, Stock, Production, Commandes, Utilisateurs
4. Les KPI et analyses	Indicateurs de base et avancés
5. La boutique en ligne (shop)	Catalogue public, panier, paiement, compte
6. Paiement en ligne — Stripe	Flux réel, webhook, idempotence
7. Comptes, rôles et sécurité	Création d'utilisateurs, blocage, suppression
8. Qualité : tests automatisés	Unitaires et intégration, commandes de démo
9. Recherche et filtres côté serveur	Stock, Production, Commandes, Utilisateurs
10. Identité visuelle	Logo et charte graphique
11. Bugs réels rencontrés et corrigés	Étude de cas techniques
12. Déploiement et code source	Dépôt Git privé
13. Limites connues et perspectives	Ce qui reste à faire
14. Conclusion	Bilan du stage

1. Contexte et objectifs

L'entreprise

EURL Plasti Matic est une entreprise basée à **Bordj Bou Arreridj (Algérie)**, spécialisée dans la fabrication de **vêtements de travail**, d'**équipements de protection individuelle (EPI)** et de **chaussures de sécurité industrielles**. L'entreprise gérait auparavant sa production et son stock de façon artisanale, sans outil numérique centralisé, et ne disposait d'aucune présence de vente en ligne.

Le cahier des charges du stage

Deux besoins distincts à couvrir avec une seule base technique commune :



Un tableau de bord interne

Outil réservé au personnel (administrateurs et employés) pour piloter le catalogue produit, le stock, la production et les commandes, avec des indicateurs de performance (KPI) simples et avancés pour aider à la décision.



Une boutique en ligne

Vitrine publique permettant à un client de parcourir le catalogue, ajouter au panier, créer un compte (ou commander en invité) et payer réellement en ligne.

Choix d'architecture : plutôt que deux projets isolés avec du code dupliqué, les deux applications partagent la **même base de données**, la **même API** et les **mêmes schémas de validation** — un mouvement de stock déclenché par une vente sur la boutique est immédiatement visible dans le tableau de bord, sans synchronisation manuelle.

Périmètre fonctionnel livré

- Gestion du catalogue produits (produits, variantes taille/couleur, prix, coût, images)
- Gestion de stock avec ledger de mouvements (entrée / sortie / ajustement)
- Suivi de production (lots de fabrication, rendement, taux de défauts)
- Gestion des commandes et de leur statut
- Gestion des utilisateurs internes et clients, avec rôles et sécurité
- Tableaux de KPI simples et avancés avec visualisation de données
- Boutique en ligne complète avec paiement réel (Stripe, mode test)
- Tests automatisés (unitaires et intégration) sur le back-end et les deux front-ends

2. Architecture technique

Stack technologique

Couche	Technologies
Back-end (API)	NestJS · Prisma ORM · PostgreSQL · JWT (rôles ADMIN / STAFF / CUSTOMER) · Validation Zod (<code>nestjs-zod</code>) · Documentation Swagger
Front-ends	React · TypeScript · Vite · TanStack Query (React Query) · shadcn/ui · Tailwind CSS · Recharts (graphiques)
Partagé	<code>packages/shared</code> — schémas Zod comme source unique de vérité (types + validation), utilisés à la fois par l'API et les deux front-ends
Paieement	Stripe Checkout (mode test) + webhooks signés
Tests	Jest (API), Vitest + React Testing Library (front-ends)
Organisation	Monorepo npm workspaces

Structure du dépôt

```
apps/  
  api/      → API NestJS (REST, /api/docs pour Swagger)  
  dashboard/ → Application interne : KPI, catalogue, stock, production, commandes  
  shop/     → Boutique publique : catalogue, panier, paiement, compte client  
packages/  
  shared/   → Schémas Zod + utilitaires, compilés en CJS (Node) et ESM (front-ends)
```

Chaque domaine métier (`auth` , `catalog` , `inventory` , `production` , `cart` , `orders` , `analytics` , `users`) est un module NestJS autonome côté API, et un dossier `features/<nom>/{api,pages,components}` côté front-end — la même organisation des deux côtés.

Pourquoi ces choix ?

- **NestJS** impose une architecture modulaire (contrôleurs / services / modules), proche de Spring ou Angular côté back-end — adapté à un projet qui grossit avec de nombreux domaines métier.
- **Prisma** génère un client TypeScript typé à partir du schéma de base de données : aucune requête SQL manuelle pour les cas simples, et les migrations sont versionnées.
- **Zod partagé** : un seul schéma de validation défini une fois est utilisé (1) comme DTO de validation côté API, (2) comme type TypeScript, et (3) comme schéma de validation de formulaire côté front — élimine toute divergence entre ce que l'API attend et ce que le formulaire envoie.
- **React Query** gère le cache, le re-fetch automatique et les états de chargement, évitant du code répétitif de gestion d'état serveur.

Modèle de données — comment Catalogue, Stock et Production se rejoignent

Ces trois sections du tableau de bord semblent indépendantes mais partagent une seule clé de jointure — `ProductVariant.id` (un SKU : une combinaison taille/couleur d'un produit) — et un seul nombre partagé : `ProductVariant.quantity` , le solde de stock réel de ce SKU.

Catalogue

Product —< ProductVariant >— variantId (FK)
sku, taille, couleur, prix, coût, seuil d'alerte

Qui référence une ProductVariant ?

├ ProductionBatch (Production)
├ OrderItem (Commandes)
└ StockMovement (Stock – saisie manuelle)

lot TERMINÉ	→ + quantité produite	└	
commande payée	→ - quantité vendue	├	ProductVariant.quantity
mouvement manuel	→ ± quantité	└	+ nouvelle ligne StockMovement

Règle d'or du projet : chaque changement de **quantity** écrit *obligatoirement*, dans la même transaction, une ligne dans le ledger **StockMovement** . Résultat : l'historique du stock est toujours reconstituable à n'importe quelle date passée — c'est ce qui permet de calculer des KPI avancés comme la rotation de stock, sans jamais désynchroniser le solde affiché de son historique.

3. Le tableau de bord (dashboard)

Application réservée au personnel — accès protégé par rôle (**ADMIN** **STAFF**), authentification JWT.

Vue d'ensemble

Page d'accueil avec les indicateurs clés (KPI de base) : valeur du stock, unités en stock, alertes de stock bas, commandes et chiffre d'affaires sur la période, panier moyen, nombre de produits/SKU.

Catalogue

- Création / modification de produits avec upload d'images réelles (JPG/PNG/WEBP/GIF, 5 Mo max)
- Gestion des variantes (taille, couleur, SKU, prix de vente, prix de revient, seuil d'alerte stock)
- Recherche et filtre par catégorie
- Activation / désactivation d'un produit (un produit avec historique — stock, commandes, production — ne peut pas être supprimé, seulement désactivé, pour ne jamais casser la traçabilité)

Stock

- Table « Niveaux de stock » : un SKU par ligne, statut **OK** / **Stock bas**
- Table « Derniers mouvements » : historique complet des entrées / sorties / ajustements
- Formulaire « Nouveau mouvement de stock » pour les corrections manuelles (réception fournisseur, casse, inventaire physique)
- Recherche et filtres (voir section 9) directement côté serveur sur les deux tables

Production

- Création d'un lot de fabrication rattaché à une variante (SKU) précise, avec quantité planifiée
- Cycle de vie du lot : **Planifié** → **En cours** → **Terminé** (ou **Annulé**)
- Saisie de la quantité produite et des défauts
- **Passer un lot à « Terminé » ajoute automatiquement la quantité produite au stock** — lien direct entre le suivi de fabrication et l'inventaire, demandé par le cahier des charges

Commandes

- Liste de toutes les commandes (boutique), avec client (ou email invité), articles, statut de paiement et de livraison
- Changement du statut de commande (Confirmée → En préparation → Expédiée → Livrée, ou Annulée)
- Recherche par numéro de commande / client / email, filtre par statut et par statut de paiement

Utilisateurs

Détaillée en section 7 (comptes, rôles et sécurité).

4. Les KPI et analyses

Deux niveaux : des indicateurs simples (comptages et sommes directes) et des indicateurs avancés qui nécessitent une logique de calcul plus poussée.

KPI de base

- Valeur totale du stock, unités totales en stock
- Nombre d'alertes de stock bas
- Commandes et chiffre d'affaires sur la période
- Panier moyen (valeur moyenne de commande)
- Nombre de produits / SKU actifs
- Production réalisée sur la période
- Répartition des commandes par statut
- Valeur du stock par catégorie
- Tendance du chiffre d'affaires

KPI avancés

- **Rotation de stock et jours de stock restant (DIO)**, reconstruits à partir du ledger de mouvements
- **Analyse ABC** : classement des produits par contribution cumulée au chiffre d'affaires (fonction fenêtrée SQL)
- **Rendement et taux de défauts** de production, par lot
- **Risque de rupture de stock** : stock actuel ÷ vitesse de vente sur 30 jours glissants (heuristique assumée, pas une prévision statistique)
- **Tendance des ventes** sur moyenne mobile 7 jours
- Meilleurs / moins bons produits, délai de traitement des commandes

Choix technique : les KPI simples utilisent les fonctions natives de Prisma (`aggregate` , `groupBy`). Les KPI avancés qui nécessitent des fonctions fenêtrées SQL ou des calculs multi-étapes (rotation de stock, analyse ABC) utilisent des requêtes `$queryRaw` paramétrées — Prisma ne permet pas d'exprimer ce type de logique via son constructeur de requêtes standard.

5. La boutique en ligne (shop)

Application publique, accessible sans compte pour parcourir et commander en invité.

Catalogue public

- Navigation par catégorie (Chaussures de sécurité, Vêtements de travail, Équipements de sécurité, Sandales), recherche, pagination **réelle côté serveur** (chaque page = une vraie requête à l'API, pas un découpage d'une liste déjà chargée)
- Badge « Nouveau » calculé dynamiquement (produit créé il y a moins de 14 jours)
- Fiche produit avec sélection taille + couleur, affichage du stock disponible

Panier et commande

- Panier persistant (compte client ou session invité), fusion automatique du panier invité lors de la connexion
- Tunnel de commande avec adresse de livraison, email invité si non connecté
- Paiement réel via Stripe Checkout (détaillé section 6)
- Historique « Mes commandes » avec statuts visuellement distincts (couleur + icône par statut)

Compte client

- Inscription / connexion avec formulaires refondus (voir section 10)
- Gestion du profil (nom, email, photo) et changement de mot de passe
- Chargements avec squelettes (« skeleton loaders ») sur toutes les pages qui récupèrent des données, pour une interface qui ne « saute » jamais pendant le chargement

6. Paiement en ligne — intégration Stripe

Paiement réel (mode test Stripe), pas une simulation interne.

Déroulé du paiement

1. Client clique « Payer »
↓
2. POST /orders/checkout
→ crée une commande statut EN_ATTENTE (le stock N'EST PAS touché ici)
→ crée une session Stripe Checkout
→ redirige le navigateur vers la page de paiement hébergée par Stripe
↓
3. Le client paie sur la page Stripe (carte test 4242 4242 4242 4242)
↓
4. Stripe envoie un webhook signé : checkout.session.completed
↓
5. L'API vérifie la signature, décrémente le stock, écrit le mouvement de stock (StockMovement), passe la commande à PAYÉE – le tout dans UNE transaction.

Pourquoi ne pas décrémente le stock à l'étape 2 ?

Si le client abandonne le paiement, la commande reste simplement « en attente » sans jamais avoir touché au stock — aucune réservation fantôme à nettoyer.

Idempotence du webhook

Stripe peut renvoyer le même événement webhook plusieurs fois (garantie « au moins une fois », pas « exactement une fois »). Le traitement est **protégé par une condition** : il ne s'exécute que si la commande est encore **PENDING**. Une deuxième réception du même événement ne fait donc rien — le stock n'est jamais décrémente deux fois pour la même vente. Vérifié manuellement en rejouant le même événement webhook.

7. Comptes, rôles et sécurité

Trois rôles

Rôle	Accès
ADMIN	Tout le tableau de bord, y compris la gestion des utilisateurs internes
STAFF	Tout le tableau de bord sauf la gestion des utilisateurs
CUSTOMER	Boutique uniquement — aucun accès au tableau de bord (vérifié aussi côté serveur, pas seulement caché dans l'interface)

Création de comptes internes par un administrateur

Un admin peut créer un compte **ADMIN** ou **STAFF** directement depuis la page Utilisateurs :

1. L'email doit obligatoirement se terminer par `@plastimatic.com` — sinon message d'erreur explicite (« seules les adresses de l'entreprise peuvent être utilisées »)
2. Un mot de passe temporaire est généré automatiquement (12 caractères aléatoires) et affiché **une seule fois** à l'écran, avec bouton de copie — jamais stocké en clair ni ré-affichable ensuite
3. Le nouveau compte est marqué « changement de mot de passe requis » : à la première connexion, l'utilisateur est bloqué sur la page « Mon compte » tant qu'il n'a pas défini son propre mot de passe

Blocage et suppression d'un compte

- **Bloquer** : le compte ne peut plus se connecter, et s'il a déjà une session active, sa toute prochaine requête (dashboard ou boutique) est rejetée immédiatement — le statut du compte est revérifié en base à chaque requête authentifiée, pas seulement à la connexion
- **Supprimer** : suppression **définitive et réelle** de la ligne en base de données (pas une simple désactivation). Avant suppression, le nom et l'email du compte sont automatiquement recopiés sur ses commandes passées (`guestEmail`), afin que « Commandes » continue d'afficher qui a passé la commande même après suppression du compte. L'adresse de livraison (son propre enregistrement, indépendant du compte) reste également intacte.
- Un administrateur ne peut ni se bloquer, ni se supprimer lui-même (vérifié côté serveur)

Décision technique assumée : une suppression totalement définitive pose un vrai risque de traçabilité (un client supprimé, plus aucune trace de qui a acheté quoi). La solution retenue — recopier nom + email sur les commandes juste avant de supprimer le compte — a été choisie après avoir étudié l'impact réel sur les clés étrangères de la base (`ON DELETE SET NULL`) plutôt que d'implémenter une suppression « brute » sans réflexion.

8. Qualité : tests automatisés

Tests réels (pas de simulation) sur les quatre packages du monorepo — unitaires ET intégration, avec une vraie base de données pour les tests d'intégration de l'API.

Package	Outil	Ce qui est testé	Nb tests
<code>packages/shared</code>	Vitest	Schémas de validation (règles métier : domaine email, email invité), utilitaires de formatage	11
<code>apps/api</code> (unitaire)	Jest	Logique de service avec base de données simulée : suppression de compte, connexion, changement de mot de passe	12
<code>apps/api</code> (intégration)	Jest + Supertest	Application réelle démarrée + vraie base PostgreSQL de test : inscription, connexion, garde de rôle, création d'utilisateur	6
<code>apps/dashboard</code>	Vitest + Testing Library	Composants d'interface (badges de statut, boîte de confirmation avec vrai comportement d'ouverture/fermeture)	8
<code>apps/shop</code>	Vitest + Testing Library	Fiche produit (prix, rupture de stock, ajout au panier simulé)	5

Commandes pour lancer les tests en démonstration

```
# Depuis la racine du projet – tous les tests rapides (34 tests, sans base de données)
npm test

# Test d'intégration API avec vraie base de données PostgreSQL (6 tests)
npm run test:e2e

# Les deux ensemble
npm run test:all
```

Le test d'intégration démarre l'application NestJS **réelle** (mêmes gardes de rôle, même pipe de validation que l'application en production) contre une base `plastimatic_test` dédiée — il vérifie que le pare-feu de rôles (JWT + rôle ADMIN) fonctionne vraiment de bout en bout, pas seulement en isolation.

9. Recherche et filtres côté serveur

Chaque filtre listé ci-dessous déclenche une vraie requête à l'API avec des paramètres de requête réels (`?search=...&status=...`) — jamais un simple `.filter()` JavaScript sur une liste déjà chargée en mémoire, ce qui ne fonctionnerait plus correctement dès que la liste dépasse une seule page.

Page	Filtres disponibles
Stock — Niveaux de stock	Recherche (produit ou SKU), catégorie, statut (Stock bas / OK)
Stock — Derniers mouvements	Recherche (produit, SKU ou motif), type (Entrée / Sortie / Ajustement)
Production	Recherche (lot, produit ou SKU), catégorie, statut du lot
Commandes	Recherche (numéro, client, email), statut, statut de paiement
Utilisateurs	Recherche (nom, email), rôle

Point technique notable : le filtre « Stock bas / OK » compare deux colonnes de la même ligne (`quantity` et `lowStockThreshold`) — un cas que le générateur de requêtes standard de Prisma ne sait pas exprimer. Une requête SQL brute paramétrée (donc protégée contre l'injection SQL, comme toute requête préparée) a été utilisée spécifiquement pour ce cas.

10. Identité visuelle

Le logo officiel de l'entreprise (script manuscrit « Matic ») a été intégré à l'ensemble des deux applications : en-tête, pied de page, panneau de connexion, barre latérale du tableau de bord. Une seule image source est utilisée partout — sa variante blanche sur fond coloré est obtenue par un filtre CSS plutôt que par un second fichier exporté, pour n'avoir qu'un seul asset à maintenir.

Pages de connexion / inscription (boutique)

Repensées avec :

- Illustration de marque et arguments de confiance (livraison, paiement sécurisé, retours) sur le panneau latéral
- Champs avec icônes, bouton d'affichage/masquage du mot de passe
- Message d'erreur serveur affiché comme une vraie alerte (icône + encadré), plus un texte rouge isolé
- Panneau occupant exactement la hauteur d'écran disponible, pied de page invisible tant qu'on n'a pas fait défiler la page (mise au point précise, pas une valeur approximative)

11. Bugs réels rencontrés et corrigés

Trois exemples concrets illustrant une vraie démarche de débogage — utile pour montrer, à l'oral, la compréhension du système plutôt qu'un simple copier-coller de code.

① Un lot de production terminé ne mettait pas à jour le stock après modification

Symptôme : un lot complété avec 0 unité produite, puis modifié à 100 unités, ne changeait rien au stock ni au catalogue.

Cause : le code ne réagissait qu'au tout premier passage du statut vers « Terminé ». Une fois le lot déjà marqué terminé, modifier à nouveau la quantité produite ne déclenchait plus aucune synchronisation.

Correction : comparaison entre « ce qui est actuellement reflété dans le stock pour ce lot » et « ce qui devrait l'être maintenant », et application de la différence — fonctionne aussi bien pour la première complétion que pour une correction ultérieure, ou même si le lot repasse en cours puis termine à nouveau (sans compter deux fois la même production).

② La pagination de la boutique n'était pas réelle

Symptôme : au-delà de 100 produits actifs, certains deviendraient invisibles sans erreur ni message.

Cause : la boutique récupérait jusqu'à 100 produits en un seul appel et découpait le résultat en pages *dans le navigateur* — une fausse pagination avec un plafond caché.

Correction : chaque page cliquée déclenche désormais une vraie requête paginée au serveur — plus aucune limite cachée, quel que soit le nombre de produits.

③ Suppression définitive d'un utilisateur : préserver la traçabilité des commandes

Détaillé en section 7 — inclus ici car c'est un exemple de décision technique *anticipée* (vérification des contraintes de clé étrangère avant d'écrire le code) plutôt qu'un bug découvert après coup.

12. Déploiement et code source

- Dépôt Git **privé** : `github.com/mondher0/plasti-matic-platform`
- Fichier `.gitignore` excluant les secrets (`.env`), les dépendances (`node_modules`) et les fichiers compilés — seuls des fichiers `.env.example` (valeurs factices) sont versionnés comme modèle de configuration
- Installation reproductible en local en quelques commandes (voir `README.md` à la racine du dépôt) : installation des dépendances, démarrage de PostgreSQL via Docker, migrations Prisma, données de démonstration, puis lancement des trois applications

Comptes de démonstration

Rôle	Email	Mot de passe	Application
Admin	admin@plastimatic.dz	password123	dashboard
Staff	staff@plastimatic.dz	password123	dashboard
Client	client1@example.com ... client6@example.com	password123	shop

13. Limites connues et perspectives

Limites assumées (hors périmètre du stage)

- Un seul entrepôt / point de stock (pas de multi-site)
- Pas de coupons de réduction, avis produits ou liste de souhaits
- Pas de tâche automatique de nettoyage des paiements Stripe abandonnés
- Pas de flux de réinitialisation de mot de passe par email

Pistes d'évolution identifiées

- **Système de retours et remboursements** — gérer le retour d'un produit et le remboursement associé (statut de commande dédié, remboursement Stripe, remise en stock)
- Notifications email (confirmation de commande, expédition)
- Multi-entrepôt si l'entreprise ouvre un second site de stockage
- Export comptable des KPI (PDF / Excel)

14. Conclusion

Ce stage a permis de livrer une plateforme complète et fonctionnelle répondant aux deux besoins du cahier des charges : un outil de pilotage interne (catalogue, stock, production, commandes, KPI) et une boutique en ligne avec paiement réel, sur une base technique commune et cohérente.

Au-delà des fonctionnalités, le projet a été l'occasion de :

- Concevoir un modèle de données où chaque mouvement de stock est traçable et reconstituable dans le temps
- Intégrer un vrai fournisseur de paiement externe (Stripe) avec gestion correcte de l'asynchronisme (webhooks) et de l'idempotence
- Mettre en place une vraie couverture de tests automatisés, unitaires et d'intégration, sur l'ensemble du monorepo
- Déboguer des problèmes réels de synchronisation de données en environnement de production simulée
- Prendre des décisions techniques réfléchies sur des sujets sensibles (suppression de compte, sécurité des rôles) plutôt que d'implémenter la solution la plus rapide

EURL Plasti Matic — Bordj Bou Arreridj, Algérie